

Shortest Distance After Road Addition Queries I

© Authored by Arpit Raj, LNMIIT Jaipur | July 25, 2026

1. Dry Run & Intuition

Suppose initially we have:

```
0 → 1 → 2 → 3 → 4
```

Then initially, the shortest distance is **4**.

First Query: says $0 \rightarrow 3$ are connected.

Then the shortest path becomes:

```
0 → 3 → 4
```

The shortest distance is now **2**.

Next Query: $1 \rightarrow 4$.

Then we have two potential paths:

```
0 → 1 → 4  
or  
0 → 3 → 4
```

Both yield a shortest distance of **2**.



Observation: ∵ every edge has weight = 1 → use BFS.

2. Complexity Analysis

Time Complexity per BFS

$$O(N + E)$$

Total Time for Q Queries

$$O(Q \times (N + E))$$

3. Algorithm Steps

- S1.** Prepare the initial adjacency list for $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \dots \rightarrow n-1$.
- S2.** For every query run BFS. For every query make the connection, prepare a queue, prepare a distance matrix, and initialize everything to INT_MAX.
- S3.** As soon as you visit a node for the first time, update its distance and push to queue.
- S4.** Once queue is empty, you got the answer for that query.

4. Implementation (C++)

```
class Solution {
public:
    vector<int> shortestDistanceAfterQueries(int n, vector<vector<int>>& queries) {

        //step1 intital adjacency matrix bana lenge for
        //0->1->2->3->...n-1

        vector<vector<int>> adj(n);
        for(int i=0;i<n-1;i++){
            adj[i].push_back(i+1);
        }

        vector<int> ans;

        //step2 ab har query run karenge new road bnayenge and bfs krenge to see shortest d

        for(auto &newroad:queries){
            int u=newroad[0];
            int v=newroad[1];

            adj[u].push_back(v);

            //ek ek krke answer dena he to isliye har baar new sadak banane ke baad cost d
```

```

        queue<int> q;
        vector<int> dist(n, INT_MAX);

        q.push(0);
        dist[0]=0;

        while(!q.empty()){
            int currnode=q.front();
            q.pop();

            for(auto &newnode: adj[currnode]){
                if(dist[newnode]==INT_MAX){ //pehli baar visit krne pe hi update kreng
                                                //kyunki ye shortest path guarentee krega

                    dist[newnode]=dist[currnode]+1;
                    q.push(newnode);
                }
            }

            ans.push_back(dist[n-1]);
        }

        return ans;
    }
};

```